

JavaScript

Professional Complete Guide

ES6+ · DOM · Events · Async · Classes · Modules · Regex · WeakMap · Proxy · Testing

Book 4 of 15 · W3Schools Professional Series · 2026

Table of Contents

#	Chapter	Key Topics
1	Fundamentals	Variables, types, operators, coercion, strict mode
2	Control Flow & Loops	if/switch, all loops, iterators, generators
3	Functions Deep Dive	Closures, IIFE, higher-order, currying, memoize
4	Arrays & Objects	All methods, destructuring, spread, optional chaining
5	ES6+ Features	Classes, symbols, iterators, generators, modules
6	DOM & BOM	Selection, manipulation, events, forms, storage
7	Async JavaScript	Callbacks, Promises, async/await, fetch, AbortController
8	Error Handling & Regex	try/catch, custom errors, RegExp API
9	Advanced JS	Proxy, Reflect, WeakMap, WeakSet, WeakRef, FinalizationRegistry
10	Modules & Tooling	import/export, dynamic import, bundlers, npm
11	Testing & Patterns	Jest basics, design patterns, functional programming
12	Exercises	20 exercises covering all JavaScript topics

Chapter 1: Fundamentals

Variables, types, coercion, operators

Variable Declarations

```
// var – function-scoped, hoisted, avoid in modern JS
// let – block-scoped, reassignable, NOT hoisted cleanly
// const – block-scoped, cannot reassign binding (but object props can change)

const PI = 3.14159; // primitive: truly immutable binding
const user = { name: "Alice" };
user.name = "Bob"; // OK – mutating the object
// user = {}; // ERROR – reassigning binding

// Temporal Dead Zone (TDZ)
// console.log(x); // ReferenceError – TDZ
let x = 5;

// Hoisting behaviour
console.log(typeof foo); // "undefined" – var hoisted
var foo = 42;
// console.log(typeof bar); // ReferenceError – let/const TDZ
let bar = 99;
```

Types & Coercion

```
// Primitive types: string, number, bigint, boolean, undefined, null, symbol
// Reference types: object, array, function, date, regexp, map, set...

// typeof
typeof "hello" // "string"
typeof 42 // "number"
typeof true // "boolean"
typeof undefined // "undefined"
typeof null // "object" ← famous JS bug
typeof {} // "object"
typeof [] // "object" ← use Array.isArray()
typeof function(){} // "function"
typeof Symbol() // "symbol"
typeof 42n // "bigint"

// Explicit conversion
String(42) // "42"
Number("3.14") // 3.14
Number("") // 0
Number("abc") // NaN
Number(true) // 1
Number(null) // 0
Number(undefined) // NaN
Boolean(0) // false
Boolean("") // false
Boolean("0") // true ← gotcha!
Boolean([]) // true ← gotcha!
parseInt("3.9px") // 3
parseFloat("3.9px") // 3.9

// Loose vs strict equality
0 == false // true (coercion)
0 === false // false (no coercion)
"" == false // true
null == undefined // true
null === undefined // false
NaN == NaN // false ← NaN is never equal
Number.isNaN(NaN) // true ← correct check
Object.is(NaN, NaN) // true
Object.is(0, -0) // false
```

Chapter 2: Functions Deep Dive

Closures, currying, memoization, patterns

Closures

```
// A closure remembers the scope it was created in
function makeCounter(initial = 0) {
  let count = initial;
  return {
    increment() { return ++count; },
    decrement() { return --count; },
    reset() { count = initial; },
    value() { return count; },
  };
}
const counter = makeCounter(10);
counter.increment(); // 11
counter.increment(); // 12
counter.reset(); // 10

// Classic closure bug and fix
// BUG: all setTimeout share same i (var)
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 100); // 3 3 3
}

// FIX 1: use let (block scope)
for (let i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 100); // 0 1 2
}

// FIX 2: IIFE to capture
for (var i = 0; i < 3; i++) {
  ((j) => setTimeout(() => console.log(j), 100))(i);
}
```

Currying & Memoization

```
// Currying – convert f(a,b,c) to f(a)(b)(c)
const curry = fn => {
  const arity = fn.length;
  return function curried(...args) {
    if (args.length >= arity) return fn(...args);
    return (...more) => curried(...args, ...more);
  };
};

const add = curry((a, b, c) => a + b + c);
add(1)(2)(3); // 6
add(1, 2)(3); // 6
add(1)(2, 3); // 6

// Memoization – cache results
function memoize(fn) {
  const cache = new Map();
  return function(...args) {
    const key = JSON.stringify(args);
    if (cache.has(key)) return cache.get(key);
    const result = fn.apply(this, args);
    cache.set(key, result);
    return result;
  };
}

const expensiveFib = memoize(function fib(n) {
  if (n <= 1) return n;
  return expensiveFib(n-1) + expensiveFib(n-2);
});
expensiveFib(40); // fast!
```

Chapter 3: Arrays & Objects

All methods, destructuring, spread, optional chaining

Array Methods Complete

```
const nums = [1, 2, 3, 4, 5];

// Iteration
nums.forEach((v,i) => console.log(i,v));
nums.map(v => v * 2); // [2,4,6,8,10] – new array
nums.filter(v => v % 2 === 0); // [2,4]
nums.reduce((acc,v) => acc+v, 0); // 15
nums.reduceRight((acc,v)=>acc+v); // same but right-to-left

// Search
nums.find(v => v > 3); // 4 (first match)
nums.findIndex(v => v > 3); // 3 (index of first match)
nums.findLast(v => v < 4); // 3 (last match)
nums.findLastIndex(v => v < 4); // 2
nums.includes(3); // true
nums.indexOf(3); // 2
nums.some(v => v > 4); // true (any match)
nums.every(v => v > 0); // true (all match)

// Transformation
[1,[2,[3]]].flat(); // [1,2,[3]]
[1,[2,[3]]].flat(Infinity); // [1,2,3]
nums.flatMap(v => [v, v*2]); // [1,2,2,4,3,6,4,8,5,10]
nums.slice(1, 3); // [2,3] non-mutating
[...nums].reverse(); // [5,4,3,2,1] (copy first!)
[...nums].sort((a,b) => a-b); // ascending

// Mutation
nums.push(6); nums.pop();
nums.unshift(0); nums.shift();
nums.splice(2, 1, 99); // remove 1 at idx 2, insert 99
nums.fill(0, 2, 4); // fill 0 from idx 2 to 4
nums.copyWithin(0, 3); // copy from idx3 to start

// Creation
Array.from("hello"); // ["h","e","l","l","o"]
Array.from({length:5}, (_,i)=>i); // [0,1,2,3,4]
Array.of(1,2,3); // [1,2,3]
[...new Set([1,1,2,2,3])]; // [1,2,3] unique
```

Object Methods & Patterns

```

const user = { id:1, name:"Alice", age:30, city:"Berlin" };

// Iteration
Object.keys(user); // ["id","name","age","city"]
Object.values(user); // [1,"Alice",30,"Berlin"]
Object.entries(user); // [[id,1],[name,Alice],...]
Object.fromEntries([["a",1],["b",2]]); // {a:1,b:2}

// Clone & merge
const copy = { ...user };
const extended = { ...user, role: "admin" };
const merged = Object.assign({}, user, { role:"admin" });

// Deep clone (modern)
const deep = structuredClone(user);

// Property descriptors
Object.defineProperty(user, "ssn", {
  value: "123-45-6789",
  writable: false, // cannot change value
  enumerable: false, // hidden from for..in
  configurable: false, // cannot delete or redefine
});

// Freeze & seal
Object.freeze(user); // no changes at all
Object.seal(user); // can change values, not structure

// Optional chaining
const zip = user?.address?.zip; // undefined (no error)
const fn = user?.greet?.(); // call if exists
const val = arr?.[0]?.name; // array item

// Nullish coalescing
const name = user.name ?? "Anonymous";
const score = user.score ?? 0;

// Logical assignment (ES2021)
user.name ||= "Anonymous"; // assign if falsy
user.score &&= user.score * 2; // assign if truthy
user.config ??= {}; // assign if null/undefined

```

Chapter 4: ES6+ Classes & Modules

Classes, symbols, iterators, generators, modules

Classes

```
class EventEmitter {
  #listeners = new Map(); // private field (ES2022)

  on(event, fn) {
    if (!this.#listeners.has(event))
      this.#listeners.set(event, new Set());
    this.#listeners.get(event).add(fn);
    return this; // fluent
  }

  off(event, fn) {
    this.#listeners.get(event)?.delete(fn);
    return this;
  }

  emit(event, ...args) {
    this.#listeners.get(event)?.forEach(fn => fn(...args));
    return this;
  }

  once(event, fn) {
    const wrapper = (...args) => { fn(...args); this.off(event, wrapper); };
    return this.on(event, wrapper);
  }

  static create() { return new EventEmitter(); }
}

class Store extends EventEmitter {
  #state;
  constructor(initialState = {}) {
    super();
    this.#state = initialState;
  }
  setState(updates) {
    this.#state = { ...this.#state, ...updates };
    this.emit("change", this.#state);
  }
  get state() { return { ...this.#state }; } // immutable copy
}
```

Generators & Iterators


```
// Custom iterator protocol
const range = {
  from: 1, to: 5,
  [Symbol.iterator]() {
    let current = this.from;
    const last = this.to;
    return {
      next() {
        return current <= last
          ? { value: current++, done: false }
          : { value: undefined, done: true };
      }
    };
  }
};
console.log(...range); // [1,2,3,4,5]

// Generator function
function* fibonacci() {
  let [a, b] = [0, 1];
  while (true) {
    yield a;
    [a, b] = [b, a + b];
  }
}

const fib = fibonacci();
Array.from({length: 10}, () => fib.next().value);
// [0,1,1,2,3,5,8,13,21,34]

// Generator as async flow control
function* steps() {
  const user = yield fetchUser(1);
  const posts = yield fetchPosts(user.id);
  const comments = yield fetchComments(posts[0].id);
  return { user, posts, comments };
}
```

Chapter 5: Async JavaScript

Promises, async/await, fetch, AbortController

Promises Deep Dive

```
// Promise states: pending → fulfilled | rejected
const p = new Promise((resolve, reject) => {
  setTimeout(() => {
    Math.random() > 0.5 ? resolve("success") : reject(new Error("fail"));
  }, 1000);
});

// Chaining
p.then(data => transform(data))
  .then(result => save(result))
  .catch(err => console.error(err))
  .finally(() => cleanup());

// Promise combinators
Promise.all([p1, p2, p3]) // all resolve or first rejects
  .then(([r1, r2, r3]) => { });

Promise.allSettled([p1, p2, p3]) // wait for ALL regardless
  .then(results => {
    results.forEach(r => {
      if (r.status === "fulfilled") console.log(r.value);
      else console.error(r.reason);
    });
  });

Promise.race([p1, p2, p3]) // first to settle wins
Promise.any([p1, p2, p3]) // first to FULFILL wins

// Promisify a callback-based function
const readFile = (path) => new Promise((resolve, reject) => {
  fs.readFile(path, "utf8", (err, data) => {
    if (err) reject(err); else resolve(data);
  });
});
```

async/await & Fetch API

```

// async/await – syntactic sugar over Promises
async function fetchUser(id) {
  const response = await fetch(`/api/users/${id}`);
  if (!response.ok)
    throw new Error(`HTTP ${response.status}`);
  return response.json();
}

// Error handling patterns
async function safeLoad(id) {
  try {
    const user = await fetchUser(id);
    return { data: user, error: null };
  } catch (err) {
    return { data: null, error: err.message };
  }
}

// Parallel execution
async function loadDashboard(userId) {
  const [user, posts, notifications] = await Promise.all([
    fetchUser(userId),
    fetchPosts(userId),
    fetchNotifications(userId),
  ]);
  return { user, posts, notifications };
}

// AbortController – cancel requests
async function fetchWithTimeout(url, ms = 5000) {
  const controller = new AbortController();
  const timeoutId = setTimeout(() => controller.abort(), ms);
  try {
    const res = await fetch(url, { signal: controller.signal });
    return await res.json();
  } catch (err) {
    if (err.name === "AbortError") throw new Error("Request timed out");
    throw err;
  } finally {
    clearTimeout(timeoutId);
  }
}

// Full fetch with options
const response = await fetch("/api/users", {
  method: "POST",
  headers: { "Content-Type": "application/json",
    "Authorization": `Bearer ${token}` },
  body: JSON.stringify({ name: "Alice", age: 30 }),
  signal: controller.signal,
});
const data = await response.json();

```

Chapter 6: DOM & Events

Selecting, manipulating, events, delegation

DOM Selection & Manipulation

```
// Modern selection
document.querySelector(".card"); // first match
document.querySelectorAll(".card"); // NodeList
document.getElementById("hero");
document.getElementsByClassName("btn"); // live HTMLCollection
element.closest(".container"); // up the tree
element.matches(":hover"); // true/false

// Content
el.textContent = "Safe text"; // XSS-safe
el.innerHTML = "<b>Bold</b>"; // parse HTML
el.innerText = "Rendered text"; // respects CSS
el.outerHTML; // element + self

// Attributes
el.setAttribute("data-id", "42");
el.getAttribute("data-id"); // "42"
el.removeAttribute("disabled");
el.dataset.userId = "42"; // data-user-id

// Classes
el.classList.add("active", "visible");
el.classList.remove("hidden");
el.classList.toggle("open");
el.classList.replace("old", "new");
el.classList.contains("active"); // true/false

// Styles
el.style.setProperty("--color", "red"); // CSS variable
el.style.backgroundColor = "#264DE4";
getComputedStyle(el).fontSize; // actual computed value

// DOM creation & insertion
const card = document.createElement("div");
card.className = "card";
card.textContent = "Hello";

parent.appendChild(card); // add at end
parent.prepend(card); // add at start
parent.insertBefore(card, sibling);
sibling.insertAdjacentElement("afterend", card);
el.replaceWith(newEl);
el.remove();

// Template
const tpl = document.querySelector("#card-template");
const clone = tpl.content.cloneNode(true);
document.body.appendChild(clone);
```

Events

```
// addEventListener
btn.addEventListener("click", handler);
btn.addEventListener("click", handler, { once: true });
btn.addEventListener("click", handler, { passive: true }); // scroll perf
btn.removeEventListener("click", handler);

// Event object
el.addEventListener("click", (e) => {
  e.preventDefault(); // stop default (form submit, link)
  e.stopPropagation(); // stop bubbling
  e.stopImmediatePropagation(); // stop all handlers
  console.log(e.target); // element clicked
  console.log(e.currentTarget); // element with listener
  console.log(e.type); // "click"
  console.log(e.key); // keyboard key
  console.log(e.clientX, e.clientY); // mouse position
});

// Event delegation – one listener for many elements
document.querySelector("ul").addEventListener("click", (e) => {
  const li = e.target.closest("li");
  if (!li) return;
  const action = li.dataset.action;
  console.log("Action:", action);
});

// Custom events
const event = new CustomEvent("userLoggedIn", {
  bubbles: true,
  cancelable: true,
  detail: { userId: 42, name: "Alice" }
});
document.dispatchEvent(event);

document.addEventListener("userLoggedIn", (e) => {
  console.log("User:", e.detail.name);
});
```

Chapter 7: Advanced JS

Proxy, Reflect, WeakMap, WeakSet, Symbol

Proxy & Reflect

```
// Proxy – intercept object operations
const handler = {
  get(target, prop, receiver) {
    console.log(`Getting ${prop}`);
    return Reflect.get(target, prop, receiver);
  },
  set(target, prop, value, receiver) {
    if (prop === "age" && typeof value !== "number")
      throw new TypeError("age must be a number");
    return Reflect.set(target, prop, value, receiver);
  },
  has(target, prop) {
    return prop in target;
  },
  deleteProperty(target, prop) {
    if (prop.startsWith("_"))
      throw new Error(`Cannot delete private ${prop}`);
    return Reflect.deleteProperty(target, prop);
  }
};

const user = new Proxy({ name: "Alice", age: 30 }, handler);
user.name; // logs "Getting name", returns "Alice"
user.age = "old"; // throws TypeError

// Proxy for validation (reactive-like)
function reactive(obj, onChange) {
  return new Proxy(obj, {
    set(target, prop, value) {
      const old = target[prop];
      target[prop] = value;
      if (old !== value) onChange(prop, value, old);
      return true;
    }
  });
}
```

WeakMap, WeakSet & Symbols

```

// WeakMap – keys must be objects, GC-friendly
const cache = new WeakMap();

function process(obj) {
  if (cache.has(obj)) return cache.get(obj);
  const result = expensiveComputation(obj);
  cache.set(obj, result); // auto-deleted when obj is GCd
  return result;
}

// WeakMap for private data (pre-class-fields pattern)
const _private = new WeakMap();
class Counter {
  constructor() { _private.set(this, { count: 0 }); }
  increment() { _private.get(this).count++; }
  get value() { return _private.get(this).count; }
}

// WeakSet – track objects without preventing GC
const seen = new WeakSet();
function processOnce(obj) {
  if (seen.has(obj)) return;
  seen.add(obj);
  doProcess(obj);
}

// Symbol – unique identifiers
const id = Symbol("id");
const id2 = Symbol("id");
id === id2; // false – always unique

// Well-known Symbols
class MyArray {
  [Symbol.iterator]() { /* custom iterator */ }
}

class Money {
  constructor(amount, currency) {
    this.amount = amount; this.currency = currency;
  }
  [Symbol.toPrimitive](hint) {
    if (hint === "number") return this.amount;
    return `${this.amount} ${this.currency}`;
  }
}
const price = new Money(42, "EUR");
+price; // 42 (numeric hint)
`${price}`; // "42 EUR" (string hint)

```

Chapter 8: Error Handling & Regex

Custom errors, RegExp API complete

Error Handling

```
// Error hierarchy
// Error
// ■■■ TypeError – wrong type
// ■■■ RangeError – value out of range
// ■■■ ReferenceError – undefined variable
// ■■■ SyntaxError – parse error
// ■■■ URIError – bad URI
// ■■■ EvalError – eval() error

// Custom errors
class AppError extends Error {
  constructor(message, code, meta = {}) {
    super(message);
    this.name = "AppError";
    this.code = code;
    this.meta = meta;
    this.timestamp = new Date().toISOString();
    if (Error.captureStackTrace)
      Error.captureStackTrace(this, AppError);
  }
}

class HttpError extends AppError {
  constructor(status, message) {
    super(message, `HTTP_${status}`);
    this.status = status;
    this.name = "HttpError";
  }
}

// Result pattern (functional error handling)
function tryCatch(fn) {
  try {
    return { ok: true, value: fn() };
  } catch (err) {
    return { ok: false, error: err };
  }
}

const result = tryCatch(() => JSON.parse(maybeInvalidJson));
if (result.ok) use(result.value);
else log(result.error);
```

Regular Expressions


```

// Creating regex
const re1 = /pattern/flags;
const re2 = new RegExp("pattern", "flags");

// Flags
// g – global (find all)
// i – case-insensitive
// m – multiline (^ and $ match line boundaries)
// s – dotAll (. matches newlines)
// u – unicode
// d – indices (match.indices)
// v – unicode sets (ES2024)

// Methods
/abc/.test("abcdef"); // true
"hello".match(/l+/g); // ["ll"]
"hello".matchAll(/l/g); // iterator of all matches
"hello world".replace(/o/g, "0"); // "hell0 w0rld"
"a1b2c3".replaceAll(/d/g, "#"); // "a#b#c#"
"a,b,,c".split(/,+/); // ["a","b","c"]
"abc".search(/b/); // 1 (index)

// Capture groups
const date = "2026-05-10";
const m = date.match(/^(?<y>\d{4})-(?<m>\d{2})-(?<d>\d{2})$/);
m.groups.y; // "2026" m.groups.m; // "05"

// Lookahead & lookbehind
/\d+(?= dollars)/.test("100 dollars"); // "100"
/(?<=\$)\d+/.test("$100"); // "100"
/\d+(?! dollars)/.test("100 euros"); // "100"
/(?<!\$)\d+/.test("€100"); // "100"

// Replace with function
"hello world".replace(/(\w+)/g, (match) => match.toUpperCase());
// "HELLO WORLD"

```

Chapter 9: Exercises

20 professional JavaScript exercises

Exercises

FILL

Ex 1. What is the output of: `typeof null`?

■ **Answer:** "object" – This is a historical bug in JavaScript that was never fixed for backward compatibility reasons.

WRITE

Ex 2. Write a debounce function that delays execution until after N ms of silence.

■ **Answer:**

```
function debounce(fn,ms){let t;return(...args)=>{clearTimeout(t);t=setTimeout(()=>fn(...args),ms);};}
```

TRUE/FALSE

Ex 3. `const` prevents mutation of object properties. (True/False)

■ **Answer:** FALSE – `const` prevents reassigning the binding, but object properties can still be mutated. Use `Object.freeze()` for deep immutability.

WRITE

Ex 4. Write a throttle function that limits execution to once per N ms.

■ **Answer:**

```
function throttle(fn,ms){let last=0;return(...a)=>{const now=Date.now();if(now-last>=ms){last=now;return fn(...a);};};}
```

EXPLAIN

Ex 5. What is event delegation and why is it useful?

■ **Answer:** Event delegation attaches ONE listener to a parent instead of many on children. Benefits: fewer listeners (memory), works for dynamically added elements, easier maintenance. Uses event bubbling + `e.target.closest()`.

WRITE

Ex 6. Write a deep clone function without using `JSON.stringify`.

■ **Answer:**

```
function deepClone(val){if(val===null||typeof val!=='object')return val;if(val instanceof Date)return new Date(val);if(Array.isArray(val))return val.map(deepClone);return Object.fromEntries(Object.entries(val).map(([k,v])=>[k,deepClone(v)]));}
```

FILL

Ex 7. What is the difference between Promise.all and Promise.allSettled?

■ **Answer:** Promise.all: rejects immediately if any promise rejects. Promise.allSettled: waits for ALL and returns array of {status,value/reason} – never rejects.

WRITE

Ex 8. Write a function that retries a failed async operation N times.

■ **Answer:**

```
async function retry(fn,times=3,delay=1000){try{return await fn();}catch(e){if(times===1)throw e;await new Promise(r=>setTimeout(r,delay));return retry(fn,times-1,delay);}}
```

EXPLAIN

Ex 9. Explain the difference between == and === in JavaScript.

■ **Answer:** === (strict): compares value AND type – no coercion. == (loose): performs type coercion first. Always use === to avoid bugs like 0=='0' (true), null==undefined (true), false==0 (true).

WRITE

Ex 10. Write a simple Observable/pub-sub implementation.

■ **Answer:**

```
class EventBus{#events=new Map();on(e,fn){(this.#events.get(e)||this.#events.set(e,new Set())).add(fn);return( )=>this.#events.get(e)?.delete(fn);}emit(e,...a){this.#events.get(e)?.forEach(f=>f(...a));}}
```

FILL

Ex 11. What does the optional chaining operator ?. do?

■ **Answer:** Returns undefined instead of throwing if the object/property is null or undefined. Example: user?.address?.city returns undefined if user or address is null/undefined.

WRITE

Ex 12. Write a function pipeline that chains multiple functions left to right.

■ **Answer:**

```
const pipe=(...fns)=>x=>fns.reduce((v,f)=>f(v),x); const process=pipe(trim,toLowerCase,removeSpaces);
```

EXPLAIN

Ex 13. What is the JavaScript event loop and how does it handle async?

■ **Answer:** JS is single-threaded. The event loop: (1) Execute current call stack, (2) Process all microtasks (Promises), (3) Process one macrotask (setTimeout), (4) Repeat. Promise .then() runs in microtask queue – before setTimeout.

WRITE

Ex 14. Write a WeakMap-based private data store for a class.

■ **Answer:** `const _data=new WeakMap();class Person{constructor(name,age){_data.set(this,{name,age});}getName(){return _data.get(this).name;}getAge(){return _data.get(this).age;}}`

FILL

Ex 15. What does `structuredClone()` do and how does it differ from `JSON.parse(JSON.stringify())`?

■ **Answer:** `structuredClone()` deep clones objects including: `Date`, `Map`, `Set`, `RegExp`, `ArrayBuffer`, circular references. `JSON.parse/stringify` fails on: `undefined`, functions, `Date` becomes string, circular refs throw.

WRITE

Ex 16. Write a Proxy that validates all property sets against a schema.

■ **Answer:** `function validated(obj,schema){return new Proxy(obj,{set(t,k,v){const rule=schema[k];if(rule&&typeof v!=='rule')throw new TypeError(`${k} must be ${rule}`);t[k]=v;return true;}});}`

EXPLAIN

Ex 17. What is a closure and give a practical use case.

■ **Answer:** A closure is a function that remembers the variables from its outer scope even after that scope has finished. Practical use: factory functions, private state, module pattern, memoization, partial application, event handlers with captured state.

WRITE

Ex 18. Write an async queue that processes items with concurrency limit.

■ **Answer:** `async function asyncPool(limit,items,fn){const ret=[];const running=[];for(const item of items){const p=Promise.resolve().then(()=>fn(item));ret.push(p);if(limit<=items.length){const e=p.then(()=>running.splice(running.indexOf(e),1));running.push(e);if(running.length>=limit)await Promise.race(running);}}return Promise.all(ret);}`

FILL

Ex 19. What is the purpose of `Symbol.iterator`?

■ **Answer:** `Symbol.iterator` defines the default iterator for an object, making it iterable in `for...of` loops, spread operator `[...obj]`, and `Array.from()`. Returns an object with a `next()` method.

WRITE

Ex 20. Write a function that converts callbacks to Promises (`promisify`).

■ **Answer:** `function promisify(fn){return function(...args){return new Promise((resolve,reject)=>{fn(...args,(err,result)=>{if(err)reject(err);else resolve(result);}});}};`